ML 프로젝트 포트폴리오 · 2026.08 – 09 · 4인 팀

LoL 경기는 언제 승부가 결정되는가

10:00

판정 시점 — 게임 시작 10분

게임 10분 시점의 팀 지표 13개로 승패를 판정하고 **왜**까지 설명하는 서비스.
로지스틱 회귀 하나로 정확도 0.7366 — 찍기 대비 +23.6%p.

0.7366
판정 정확도

4탭
과정 기간 공개 운영

21항목
자동 검증

담당 **데이터 분석 · 모델링 · 검증 체계 · 서비스 기능 (이제민)**

GitHub **github.com/wpalswpa/lol-win-prediction** 로컬 실행 **predict.py** Python · scikit-learn · Flask · MariaDB · Riot API

OVERVIEW

한눈에 — 문제 · 접근 · 결과 · 내 몫

문제

전적 사이트는 결과만 보여준다. "언제 갈렸고, 무엇을 했어야 했나"는 아무도 말해주지 않는다.

Kaggle 다이아 랭크 9,879판 · 10분 시점 지표

접근

설명 가능한 모델(로지스틱)을 고르고, 계수를 그대로 화면의 '이유'로 쓴다. 분할을 별도 단계로 두고 봉인.

후보 8종 비교 · 13개 차이 피처 · Pipeline 한 덩어리

결과

정확도 0.7366 (10회 평균) · 승리요인 골드 > 경험치 > 드래곤. 접전(34%)은 0.615 — 정보의 한계로 공개.

AUC 0.8150 · Brier 0.1756 · 티어 차이 확인되지 않음(p=0.407)

내 몫

분석·모델링·검증 체계 전부, 그리고 9/2 부터 서비스 기능(판정·코칭·성향·랭킹·승부예측) 구현.

lolwin 패키지 · 자동 검증 21항목 · 문서 12종 · 라이브 운영

이 프로젝트에서 보여주고 싶은 것

원인 증명	계수 이상을 파고들어 원인 증명
한계 측정	못 맞히는 구간을 실측해 공개
자기 검증	배포 전 21항목 · 틀린 값으로 시험

용어 — 골드: 게임 내 화폐 · CS: 미니언 처치 수 · 접전: 10분 골드차 1,000 미만 · 홀드아웃: 학습에 쓰지 않고 봉인한 시험 데이터

ROLE & CONTRIBUTION

4인 팀에서 내가 맡은 것

저장소 커밋 · 문서 · 코드 기준

담당 영역

분석 · 모델링	EDA → 분할 봉인 → 8종 비교 → 로지스틱 채택 → 승리요인 · 오류 분석 · 골드 제외 재학습 · PCA 확인
실험 설계	10분 vs 15분 시점 비교(프로 10,656판) · 티어 일반화 실측(아이언~다이아 1,399판 직접 수집)
검증 체계	파리티 · 수치 대조(factcheck) · 회귀 50건 · 계약 테스트 — 배포 전 21항목, 반대 방향 검증 원칙
라이브러리 · 문서	lolwin 패키지(피처 정본 · 예측 · 코칭) · README · model_card · docs 12종 · 노트북
서비스 기능 (9/2~)	판정 4분면 · 코칭 · 분 단위 궤적 · 플레이 성향 레이더 · 라인 기준선 · 랭킹 · 승부예측 · Riot API 예산 운영

팀 분담 (정직하게)

정상천	팀장 · 발표자료 · 대본 · 리허설
박예은	초기 화면(frontend · templates) 골격
임의석	초기 백엔드(app.py) · 배포 스크립트 골격
이제민	위 다섯 영역 + 9/2 이후 서비스 기능 전부

숫자로 보는 기여

커밋 162 중 151 · 서비스 탭 4개 · 검증 항목 21 · 문서 26개 수치 대조 · 티어 실측 1,399판

협업 방식 한 브랜치(team) 공유 · 사람마다 손대는 파일을 갈라 충돌 0 · 검증 통과 후에만 main 스냅샷 · 담당 부재 시 인수 범위를 문서로 남김 (docs/TEAM_WORKFLOW.md)

ARCHITECTURE

학습은 한 번, 서비스는 매번 — 잇는 건 2.2KB 파일 하나

계산은 predict.py 한 곳 · 웹에 sklearn 없음



계산은 한 곳

예측 로직은 predict.py 에만. 화면·문서는 결과를 빌려 쓴다 → 화면 확률과 모델 확률이 어긋날 길이 없다

전처리를 모델과 한 덩어리로

StandardScaler 를 Pipeline 에 넣어 저장. 서비스가 표준화를 다시 구현하지 않는다 → 학습-서비스 괴리 차단

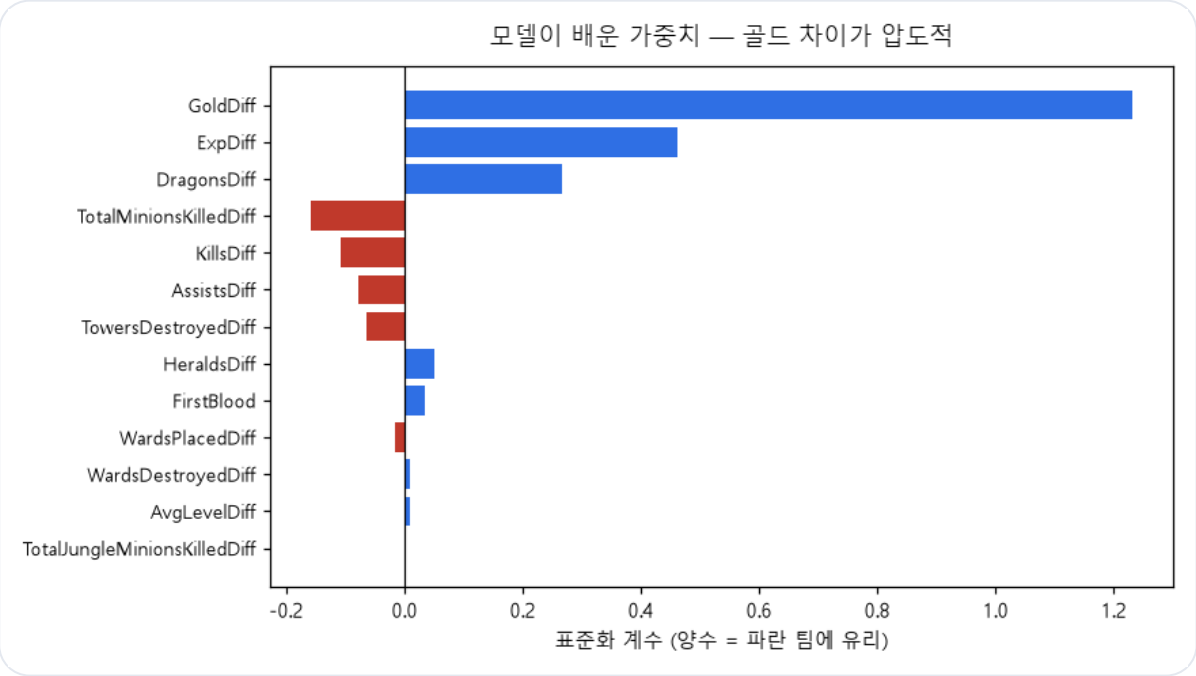
외부 의존은 스냅샷으로

랭킹·챔피언 통계·일정은 배치가 CSV 로 굳히고 서비스는 파일만 읽는다 → 사용자 요청당 외부 호출 0

CHALLENGE 1

"킬을 하면 진다"는 계수 — 버그가 아니라 단서였다

다중공선성 진단 → 검증 → 설계 반영



모델이 배운 가중치 13개 — 킬·미니언·어시스트·타워가 음수

CHALLENGE

골드가 1위인 건 예상대로. 그런데 **킬이 음수** — 킬만 따로 보면 판별력 3위인 지표가 '하면 불리하다'고 나왔다.

APPROACH

상관을 보니 성장 지표 6개가 0.9로 얽혀 있었다(킬↔골드 0.92). 골드를 이미 본 모델에게 킬은 새 정보가 아니다 — **골드를 빼고 재학습해 검증**, PCA 로 재확인(PC1 39% · 주성분 1개만 써도 0.7282).

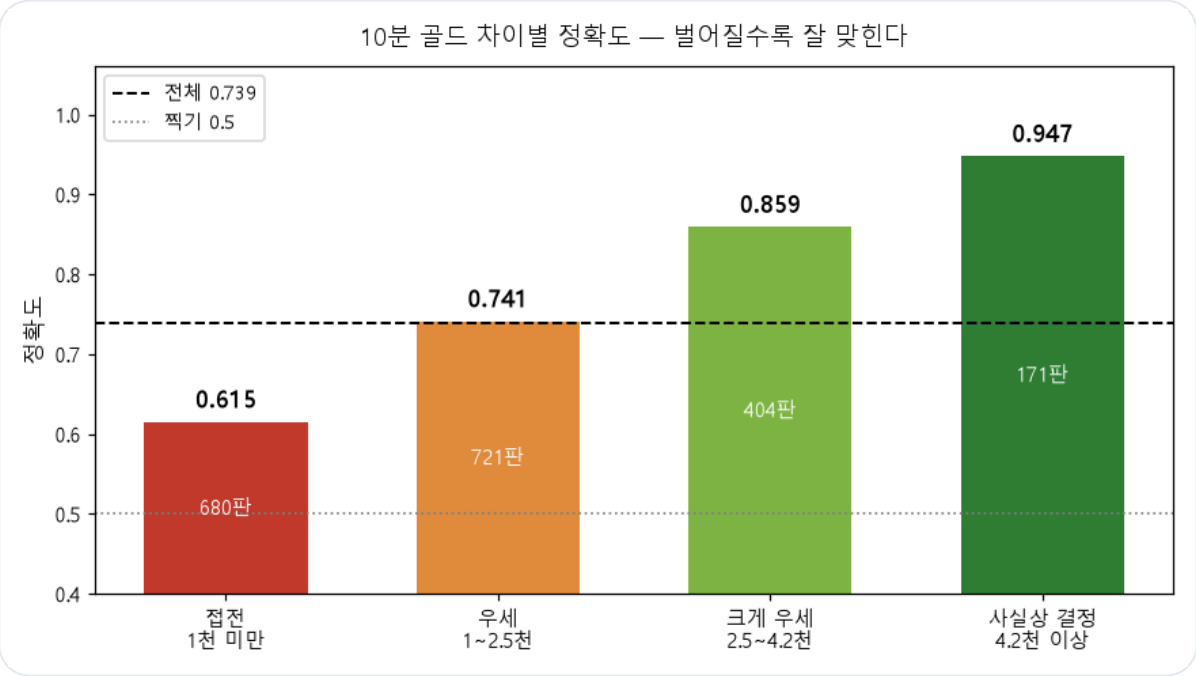
RESULT

음수 5개가 전부 양수로, 정확도는 **–0.002**. 이 발견이 서비스 코칭 설계가 됐다 — 계수를 그대로 쓰면 "CS 를 먹지 마라"가 되므로 따라오는 골드까지 함께 계산한다.

CHALLENGE 2

접전은 못 맞힌다 — 숨기는 대신 설계로 옮겼다

구간별 정확도 · 2단계 실측 · 시점 실험



골드차 구간별 정확도 — 같은 모델인데 33%p 차이

CHALLENGE

골드차 1천 미만 접전(34%)은 **0.615**, 4.2천 이상은 0.947. 접전 구간의 실제 승률이 49.5% — 10분 데이터에 단서가 없다.

APPROACH

"모델을 더 키우면?" — 접전만 따로 배우는 2단계 구조 4종을 실측. **전부 단일 모델보다 나빴다**. 대신 시점을 바꿔 봤다: 프로 경기 10,656판으로 10분 vs 15분.

RESULT

5분을 더 보면 전체 +5.25%p, 그 이득은 **접전에만 +7.8%p** (벌어진 경기는 ±0). 한계를 model_card 에 적고 화면에 "지금은 팽팽해서 자주 틀립니다" 경고를 띄운다.

CHALLENGE 3

스스로를 검증하는 파이프라인

배포 전 21항목 · 반대 방향 검증 원칙

파리티

손계산 = 모델 = 라이브 화면

0.818255 가 소수점까지 일치.
자동 테스트로 상시 대조

수치 대조

문서 26개 ↔ 근거 CSV

본문 속 모든 수치를 기계로 검증.
현재 어긋남 0건

회귀

예측 50건 골든 정답지

모델·피처가 바뀌어 결과가
달라지면 즉시 실패

계약

API 응답 ↔ 명세(serving.md)

필드·개수·타입 검사.
PUUID 유출 검사 포함

PRINCIPLE

새 검증 규칙은 틀린 값을 넣어 "실제로 잡히는지" 반대 방향으로 확인한다. 아무것도 못 잡는 규칙을 두 번 만든 뒤 세운 원칙.

RESULT

이 체계로 배포 전 회귀 세 건을 잡았다 — **화면 전체 먹통(선언 블록 스코프) · 태그 "0223" 이 숫자로 변환돼 링크 사망 · 그리드 칸 밀림.**

CHALLENGE 4

운영에서 만난 문제 4건 — 증상이 아니라 원인을 고쳤다

라이브 서비스 · 9/2 이후 단독 운영

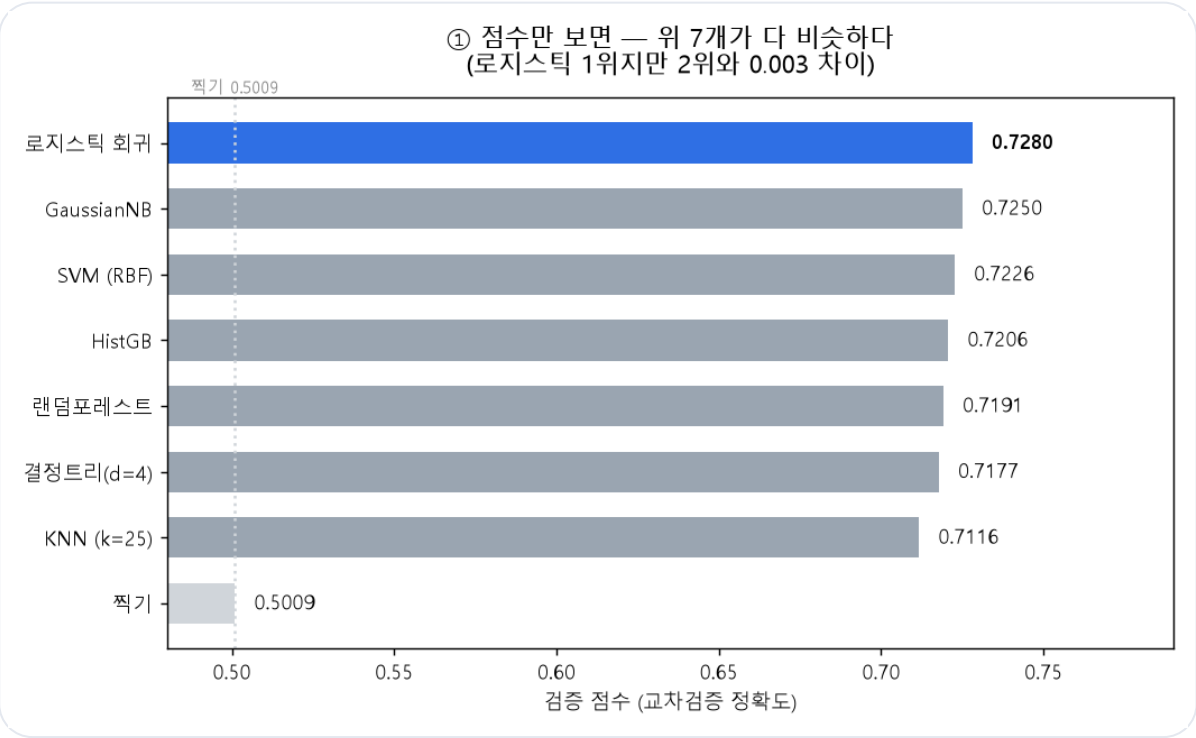
문제	원인	해결	결과
검색 한 번에 3.1초. 동시 접속하면 Riot 예산이 바닥난다	사용자 요청마다 Riot API 를 새로 호출. 같은 소환사를 다시 봐도 전부 재호출	소환사 단위 5분 캐시(_SUMMONER_CACHE, 200건 상한·만료 정리)	**3.1s → 0.02s** · 재조회는 외부 호출 0
새벽 수집기가 돌면 라이브 검색이 429 로 죽는다	수집기와 서비스가 같은 키의 예산(100회 / 120초)을 나눠 쓴다	응답 헤더의 사용량(LAST_APP_COUNT)을 읽어 70을 넘으면 수집기가 20초씩 쉰다 (RESERVE)	**사용자 몫 30 상시 확보** · 밤샘 수집 중에도 검색 정상
배포했는데 화면이 그대로다	HTML 응답에 캐시 헤더가 없어 브라우저가 옛 화면을 계속 쓴다	text/html 응답에 Cache-Control: no-store 부착(after_request 한 곳)	**배포 즉시 반영** · 로컬 = 라이브 바이트 동일 확인을 배포 절차에 추가
태그 "0223" 유저 링크가 죽고, 19자리 경기 ID 끝자리가 바뀐다	CSV → JSON 변환이 숫자처럼 보이는 열을 전부 숫자로 바꿈. JS 안전 정수(2^53) 초과	문자열로 남길 열을 명시(TEXT_COLUMNS) + 계약 테스트에 타입 검사 추가	**같은 부류 재발 차단** · 배포 전 자동 검증 21항목에 편입

공통점 — 증상을 덮는 대신 원인을 찾았고, 고친 뒤에는 같은 부류가 다시 오면 테스트가 먼저 잡도록 했다. 네 건 모두 재현 절차와 함께 docs/ 에 기록.

RESULTS

왜 로지스틱인가, 그리고 얼마나 맞히나

홀드아웃 1,976판 · 분할 10회 반복



후보 8종 교차검증 — 1위와 2위가 0.003 차이, 점수로는 못 고른다

찍기 기준선은 재는 자리마다 다르다 — 그래프 0.5009 는 교차검증, 오른쪽 표 0.5010 은 홀드아웃

선정 기준 — 점수 대신 두 가지

① 외웠는가 — 연습·검증 격차. 복잡한 모델일수록 크게 벌어졌다.

② 왜를 말할 수 있는가 — 랜덤포레스트는 중요도는 말해도 방향(+/−)을 못 말한다.

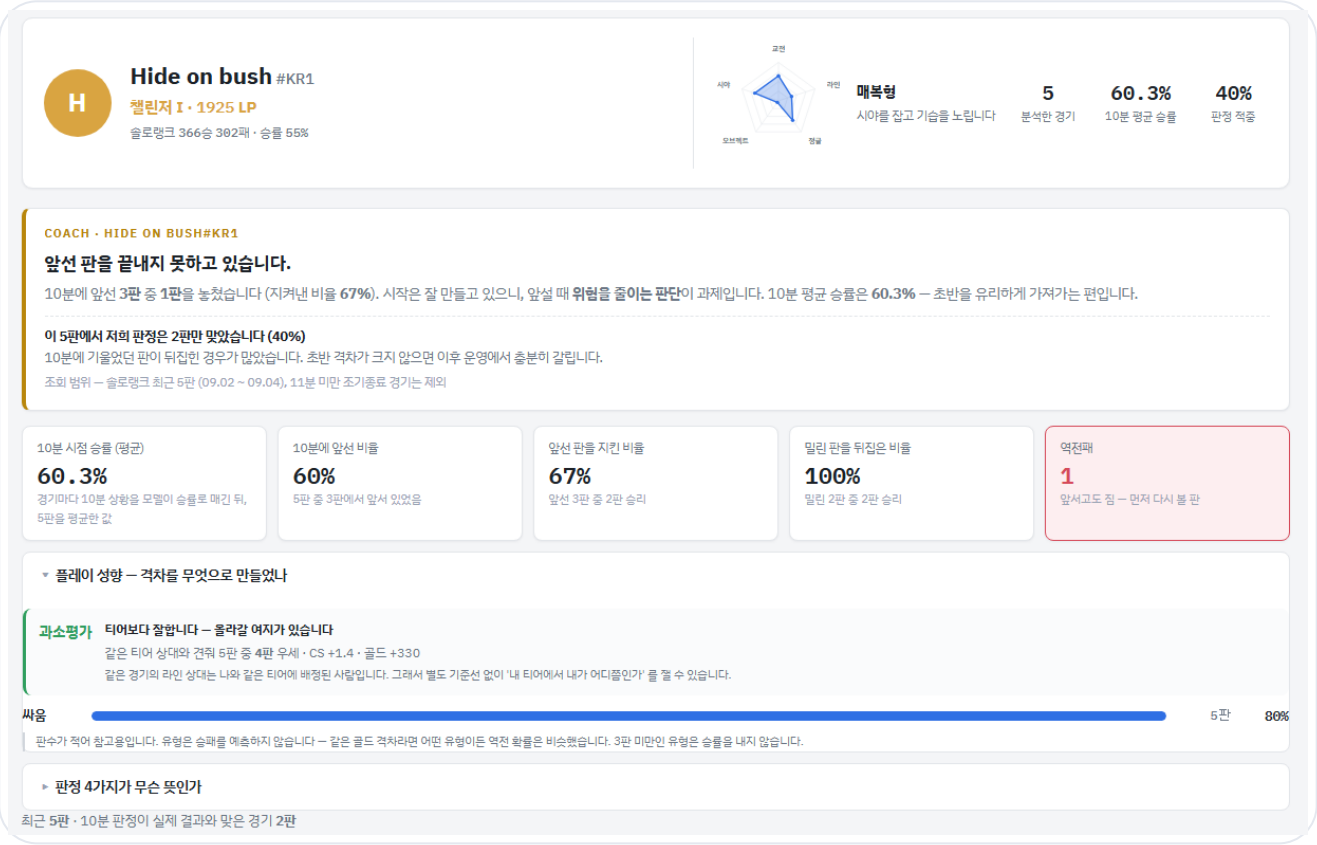
대표 정확도	**0.7366 ± 0.0081**	분할 10회 반복 평균
단일 홀드아웃	**0.7394**	시드 42 · 찍기 0.5010
AUC	**0.8150**	순위 능력
F1 / Brier	**0.7352 / 0.1756**	쏠림 · 확률 정확도
교차검증	**0.7315 ± 0.0153**	5-fold · 격차 +0.0022

일반화 — 2026년 아이언~다이아 1,399판을 직접 수집해 채점: 저티어 0.685 vs 고티어 0.707, p=0.407. "차이가 없다"가 아니라 "큰 차이는 확인되지 않았다"까지만 적었다(티어당 150~312판).

SERVICE

모델이 화면이 되기까지

탭 4개 · 2026.09 운영



판정 4분면

10분 확률(앞섬/밀림) × 실제 결과(승/패) = 우세승·역전승·역전패·열세패. 같은 패배라도 처방이 다르다.

코칭

"드래곤 하나 더였다면" — 반사실 계산. 관찰 기록이지 인과 증명이 아님을 화면에 명시.

플레이 성향 레이더

다섯 축 백분위(교전·라인·정글·오브젝트·시야). 축끼리 상관 최대 0.13이 되도록 데이터로 선정.

라인 기준선

같은 판의 라인 상대 = 같은 티어 표본. "내 티어가 거품인가"를 별도 수집 없이 잰다.

운영

Riot API 예산 100/120초 — 수집기는 30만 쓰고 양보. 5분 캐시로 동시 접속 시 외부 호출 0.

RETROSPECTIVE

배운 것, 그리고 다음

이상한 숫자는 버그가 아니라 단서다

킬 계수가 음수로 나왔을 때 모델을 의심하는 대신 데이터 구조를 진단했고, 그 발견이 서비스 설계까지 이어졌다.

한계는 숨기는 게 아니라 측정하는 것이다

접전 0.615 · 티어 p=0.407 · 2단계 4종 열위 — 못 하는 것을 실측해 문서와 화면에 적었다.

계산은 한 곳에만 둔다

예측 로직을 predict.py 하나에 두자 화면·문서·테스트가 같은 숫자를 볼 수밖에 없게 됐다.

0.7366
판정 정확도 · 10회 평균

+23.6%p
찍기(0.501) 대비

p=0.407
티어 차이 — 확인되지 않음

21항목
배포 전 자동 검증

다음 — 코칭 조언을 A/B 로 검증해 관찰을 인과로 바꾸는 것. 그때 이 서비스는 '기록'에서 '코치'가 된다.

GitHub **github.com/wpalswpa/lol-win-prediction** 로컬 실행 **predict.py**

README 에 전체 분석 과정 · docs/ 에 재현 절차 · final/ 에 산출물 6종